minHeap Operations

Nombre	Kevin Alexander Ramirez Mendez
Matrícula	A00844255
Computadora	5
Inicio del examen	2026-10-15 09:56:07
Cierre del examen	2026-10-15 10:51:32 (terminó antes del tiempo)
Reporte generado	2026-10-15 10:51:32
Calificación automática	83.33 / 100 (83.33 de 100 pts)

#	Problema	Envíos	Entregado	Veredicto	Casos	Puntos	Cuenta
1	Heap: findMin	2	10:37:02	Correcto	6/6	20 / 20	sí
2	Heap: insert	1	10:31:05	Correcto	6/6	20 / 20	sí
3	Heap: percolateDown	1	10:40:59	Correcto	6/6	20 / 20	sí
4	Heap: deleteMin	1	10:40:23	Correcto	6/6	20 / 20	sí
5	Heap: buildHeap	1	10:51:24	Respuesta incorrecta	1/6	3.33 / 20	sí
	Total					**83.33 / 100**	**83.33 / 100**

Calificación asignada por los jueces automáticos. Se toma el mejor envío de cada problema; en cada grupo cuentan los problemas con más puntos. La calificación sobre 100 es puntos obtenidos entre puntos posibles. El profesor puede revisarla.

1. Heap: findMin

Entregado: 2026-10-15 10:37:02 · Envíos: 2 · Veredicto: Correcto · Casos aprobados: 6/6 · Puntos: 20 / 20

```
 1  #include <stdexcept>
 2  #include <vector>
 3  using namespace std;
 4
 5  // Do NOT change the class: only write the body of the method at the end of the file.
 6  class BinaryHeap {
 7  public:
 8      const int &findMin() const;
 9
10  private:
11      int currentSize = 0;  // number of elements
12      vector<int> array;    // the heap, in cells 1 to currentSize; cell 0 is not used
13
14      friend struct HeapTester;
15  };
16
17  // Returns the smallest element without removing it. It is always in array[1].
18  // If the heap is empty, throws underflow_error.
19  const int &BinaryHeap::findMin() const {
20      if (currentSize == 0) throw std::underflow_error("Empty Heap");
21          return array[1];
22  }
```

2. Heap: insert

Entregado: 2026-10-15 10:31:05 · Envíos: 1 · Veredicto: Correcto · Casos aprobados: 6/6 · Puntos: 20 / 20

```
#include <vector>
using namespace std;

// Do NOT change the class: only write the body of the method at the end of the file.
class BinaryHeap {
public:
    void insert(int x);

private:
    int currentSize = 0;  // number of elements
    vector<int> array;    // the heap, in cells 1 to currentSize; cell 0 is not used

    friend struct HeapTester;
};

// Inserts x (percolate up). If the array is full, first double its size with array.resize.
// The new element starts in cell currentSize + 1 and goes up while it is smaller than its parent.
void BinaryHeap::insert(int x) {
    if (currentSize == int(array.size()) - 1){
        array.resize(array.size() * 2);
    }
    int hole = ++currentSize;                    // a hole at the end
    for (; hole > 1 && x < array[hole / 2]; hole /= 2){
        array[hole] = array[hole / 2];           // parent slides down
    }
    array[hole] = x;                             // x fits here
}
```

3. Heap: percolateDown

Entregado: 2026-10-15 10:40:59 · Envíos: 1 · Veredicto: Correcto · Casos aprobados: 6/6 · Puntos: 20 / 20

```
#include <vector>
using namespace std;

// Do NOT change the class: only write the body of the method at the end of the file.
class BinaryHeap {
private:
    int currentSize = 0;  // number of elements
    vector<int> array;    // the heap, in cells 1 to currentSize; cell 0 is not used

    friend struct HeapTester;

    void percolateDown(int hole);
};

// Moves the element of cell hole down: while it has children, compare it with the smaller one,
// and if that child is smaller, the child goes up one level. Check that the right child exists.
void BinaryHeap::percolateDown(int hole) {
    int value = array[hole];
    bool placed = false;
    while (hole * 2 <= currentSize && !placed) {
        int child = hole * 2;
        if (child != currentSize && array[child + 1] < array[child]) {
            child++;
        }
        if (array[child] < value) {
            array[hole] = array[child];
            hole = child;
```

```
28          } else {
29              placed = true;
30          }
31      }
32      array[hole] = value;
33  }
```

4. Heap: deleteMin

Entregado: 2026-10-15 10:40:23 · Envíos: 1 · Veredicto: Correcto · Casos aprobados: 6/6 · Puntos: 20 / 20

```
 1  #include <stdexcept>
 2  #include <vector>
 3  using namespace std;
 4
 5  // Do NOT change the class: only write the body of the method at the end of the file.
 6  class BinaryHeap {
 7  public:
 8      void deleteMin();
 9
10  private:
11      int currentSize = 0;  // number of elements
12      vector<int> array;    // the heap, in cells 1 to currentSize; cell 0 is not used
13
14      friend struct HeapTester;
15
16      void percolateDown(int hole);
17  };
18
19  // Already written: moves the element of cell hole down until the heap order is restored.
20  // Do not change it.
21  void BinaryHeap::percolateDown(int hole) {
22      int value = array[hole];
23      bool placed = false;
24      while (hole * 2 <= currentSize && !placed) {
25          int child = hole * 2;
26          if (child != currentSize && array[child + 1] < array[child]) {
27              child++;
28          }
29          if (array[child] < value) {
30              array[hole] = array[child];
31              hole = child;
32          } else {
33              placed = true;
34          }
35      }
36      array[hole] = value;
37  }
38
39  // Removes the smallest element. If the heap is empty, throws underflow_error.
40  // The last element moves to the root, the heap has one element less, and percolateDown(1)
     fixes it.
41  void BinaryHeap::deleteMin() {
42      if(currentSize == 0) throw underflow_error("Empty Heap");
43      array[1] = array[currentSize--];
44      percolateDown(1);
45  }
```

5. Heap: buildHeap

Entregado: 2026-10-15 10:51:24 · Envíos: 1 · Veredicto: Respuesta incorrecta · Casos aprobados: 1/6 · Puntos: 3.33 / 20

```
1  #include <vector>
2  using namespace std;
```

```
 3
 4  // Do NOT change the class: only write the body of the method at the end of the file.
 5  class BinaryHeap {
 6  private:
 7      int currentSize = 0;  // number of elements
 8      vector<int> array;    // the heap, in cells 1 to currentSize; cell 0 is not used
 9
10      friend struct HeapTester;
11
12      void buildHeap();
13      void percolateDown(int hole);
14  };
15
16  // Already written: moves the element of cell hole down until the heap order is restored.
17  // Do not change it.
18  void BinaryHeap::percolateDown(int hole) {
19      int value = array[hole];
20      bool placed = false;
21      while (hole * 2 <= currentSize && !placed) {
22          int child = hole * 2;
23          if (child != currentSize && array[child + 1] < array[child]) {
24              child++;
25          }
26          if (array[child] < value) {
27              array[hole] = array[child];
28              hole = child;
29          } else {
30              placed = true;
31          }
32      }
33      array[hole] = value;
34  }
35
36  // Puts the heap order back in cells 1 to currentSize, whatever their order: percolate down
37  // from the last parent, cell currentSize / 2, up to the root.
38  void BinaryHeap::buildHeap() {
39      // your code here
40  }
```